Design and Implementation of a 6-DOF Robotic Manipulator with Singularity-Robust Motion Control

Project Report

Anwesh Rijal (16940572)
Austin Matthew
Hochschule Ravensburg-Weingarten University

January 2026

Contents

1 Introduction **3**
1.1 Project Background . . . 3
1.1.1 Base Design Attribution . . . 3
1.2 Project Objectives . . . 3
1.2.1 Controller Objectives . . . 4
1.2.2 Software Architecture . . . 4
1.3 Scope . . . 4
1.4 Document Structure . . . 4

2 Hardware **5**
2.1 The Robot . . . 5
2.1.1 Overview . . . 5
2.1.2 Mechanical Structure . . . 5
2.1.3 Actuators . . . 6
2.1.4 Sensors . . . 7
2.1.5 End Effector . . . 7
2.2 Control Board . . . 8
2.2.1 Overview . . . 8
2.2.2 Microcontroller . . . 9
2.2.3 Motor Drivers . . . 9
2.2.4 Power System . . . 10
2.2.5 Connectors . . . 10
2.3 Electrical Connections . . . 10
2.3.1 MCU Connections . . . 10
2.3.2 Motor Driver Wiring . . . 11
2.3.3 Limit Switch Wiring . . . 12

3 MCU Firmware **13**
3.1 Firmware Overview . . . 13
3.1.1 System Architecture . . . 13
3.1.2 RTOS Tasks . . . 13
3.2 Motor Control . . . 14
3.2.1 TMC5160 Driver Interface . . . 14
3.2.2 Step Generation . . . 14
3.2.3 Mathematical Foundation . . . 14
3.2.4 The 7-Segment Profile . . . 15
3.2.5 Implementation Strategy . . . 15
3.3 Communication Protocol . . . 15

3.3.1 Framing Structure . . . 16
3.3.2 Outgoing Format (MCU → PC) . . . 16
3.3.3 Incoming Format (PC → MCU) . . . 16

4 Motion Control Software and Kinematics **17**
4.1 Kinematics Theory . . . 17
4.1.1 Forward Kinematics . . . 17
4.1.2 The Jacobian Matrix . . . 20
4.1.3 Singularity Handling: Selectively Damped Least Squares . . . 21
4.1.4 Inverse Kinematics . . . 21
4.1.5 Role in the Control Loop . . . 23
4.2 Differential IK via Quadratic Programming: A Constraint-Aware Framework 23
4.2.1 Introduction and Problem Formulation . . . 23
4.2.2 Mathematical Framework and Derivations . . . 24
4.2.3 Differential QP-Based Inverse Kinematics (OSQP Formulation) . . . 25
4.2.4 Hybrid Architecture: Topology and Global Awareness . . . 27
4.2.5 Implementation and Performance Evaluation . . . 28
4.3 Visualization and Control Interface . . . 28
4.3.1 Connection and Initialization . . . 28
4.3.2 Control Modes and Input Methods . . . 29
4.3.3 System Telemetry and Visualization . . . 32
4.4 Project Status: Limitations and Future Roadmap . . . 33
4.4.1 Hardware and Connectivity . . . 33
4.4.2 Kinematic and Dynamic Control . . . 34
4.4.3 Software and Integration . . . 34

A Source Code Repositories **36**

Chapter 1

Introduction

1.1 Project Background

This project involves the development of a 6-DOF robotic arm control system, including custom firmware and host software. The mechanical design is based on the open-source **PAROL6** desktop robot arm developed by Source Robotics.

1.1.1 Base Design Attribution

The robot hardware design used in this project is derived from:

- **PAROL6 Desktop Robot Arm** by Source Robotics
- Design by Petar Crnjak (https://github.com/PCrnjak)
- Documentation: https://source-robotics.github.io/PAROL-docs/
- Hardware: https://github.com/PCrnjak/PAROL6-Desktop-robot-arm

This project extends the base PAROL6 design with custom firmware and software implementations.

1.2 Project Objectives

The primary objectives of this project are:

1. **Build a Fully Functional 6-DOF Robotic Arm**: Assemble and integrate the mechanical structure, stepper motors, control electronics, and end effector into a working desktop manipulator.
2. **Develop Stable Motion Control Software**: Implement a robust inverse kinematics controller capable of smooth, singularity-free motion throughout the workspace.
3. **Implement Real-Time Control**: Create firmware and host software that can execute motion commands with low latency and high precision.

1.2.1 Controller Objectives

The motion controller specifically aims to:

- Provide stable velocity-level IK using quadratic programming (QP)
- Handle kinematic singularities gracefully using adaptive damping
- Support real-time interactive control via visualization interface
- Respect joint position and velocity limits at all times

1.2.2 Software Architecture

The control software consists of two main components:

- **MCU Firmware** (STM32F446): Handles low-level motor control, step generation, S-curve motion profiles, and serial communication with the host.
- **Host Software** (C++/Qt): Implements kinematics computation (FK/IK), the QP-based differential controller with singularity handling, and 3D visualization for interactive control.

1.3 Scope

This project covers:

- Mechanical assembly of the PAROL6-based robot arm
- Custom control board with STM32F446 and TMC5160 drivers
- MCU firmware for motion execution
- Host-side kinematics and motion control software
- 3D visualization and user interface

1.4 Document Structure

- **Chapter 2 - Hardware**: Robot mechanical design, control board, electrical connections
- **Chapter 3 - MCU Firmware**: Motor control, S-curve profiles, communication
- **Chapter 4 - Motion Control Software**: Kinematics, QP-based differential controller, visualization
- **Appendices**: Pinout, DH parameters, joint limits

Chapter 2

Hardware

This chapter describes the physical hardware of the our robotic arm system.

2.1 The Robot

2.1.1 Overview

Our robot is based on the **PAROL6** open-source robot arm design by Source Robotics. It is a 6 degrees of freedom (6-DOF) desktop robotic arm designed for education, research, and light industrial applications. The robot features an articulated arm configuration with all revolute joints, providing full spatial manipulation capability.

The mechanical design, including 3D printed parts and assembly, follows the original PAROL6 specifications with modifications for this project.

Parameter	**Value**
Degrees of Freedom	6 rotating joints
Weight	5.5 kg
Power Consumption	40 W
Material	3D printed ABS plastic
Communication	USB

Table 2.1: Robot Specifications

2.1.2 Mechanical Structure

The robot is constructed primarily from 3D printed ABS plastic. ABS provides good mechanical strength and thermal resistance for this application.

Joint Configuration

Joint	Function	Range	Reduction
J1	Base rotation	250°	Belt 6.4:1
J2	Shoulder lift	141°	Planetary 20:1
J3	Elbow bend	180°	Planetary 20:1 + Belt 38:42
J4	Wrist rotation	212°	Belt 4:1
J5	Wrist pitch	180°	Belt 4:1
J6	Tool rotation	∞	Planetary 10:1

Table 2.2: Joint Configuration and Reduction Ratios

Spherical Wrist Design

The robot uses a **spherical wrist** configuration, where the last three joint axes (J4, J5, J6) intersect at a single point called the wrist center. This design is common in industrial manipulators and simplifies inverse kinematics by decoupling position and orientation problems.

Kinematic Singularities

Singularities are configurations where the robot loses one or more degrees of freedom. For our 6-DOF spherical wrist arm, three main singularities exist:

1. **Wrist Singularity**: Occurs when J4 and J6 axes align (J5 = 0° or 180°)
2. **Shoulder Singularity**: Wrist center lies on the J1 axis
3. **Elbow Singularity**: Arm fully extended or folded

Near singularities, the inverse Jacobian produces very large joint velocities, causing instability. Our software mitigates this using adaptive damping techniques (see Chapter 4).

2.1.3 Actuators

All joints use stepper motors with the following configuration:

- **Motor type**: NEMA 17 / NEMA 23 stepper motors
- **Steps per revolution**: 200 (1.8° step angle)
- **Microstepping**: 32 microsteps (6400 steps/revolution)
- **Drivers**: TMC5160 (BigTreeTech) with StallGuard [7]
- **Gearboxes**: Precision planetary gearboxes and belt drives

Step Resolution

With 32 microstepping and gear reduction, the theoretical step resolution at each joint:

Joint	Resolution (rad)	Resolution (deg)
J1	0.000153	0.00879°
J2	0.000049	0.00281°
J3	0.000054	0.00311°
J4	0.000245	0.01406°
J5	0.000245	0.01406°
J6	0.000098	0.00563°

Table 2.3: Joint Step Resolution (after gear reduction)

2.1.4 Sensors

- **Position sensing**: AS5600 magnetic encoders for closed-loop position feedback
- **Limit switches**: Used for homing and end-of-travel protection

2.1.5 End Effector

The end effector is a custom-designed electric gripper based on the **SSG-48 Adaptive Electric Gripper** by Source Robotics (https://github.com/PCrnjak/SSG-48-adaptive-electric-g

Gripper Redesign

The original SSG-48 design was heavily modified for this project:

- **Actuation**: Replaced original motor with a stepper motor for precise position control
- **Position Feedback**: Added AS5600 magnetic encoder for closed-loop gripper position sensing
- **Control**: Integrated with the main control system for coordinated motion

Gripper Specifications

Parameter	Value
Actuation	Stepper motor
Position Sensor	AS5600 magnetic encoder
Control Mode	Closed-loop position
Base Design	SSG-48 (Source Robotics)

Table 2.4: Gripper Specifications

2.2 Control Board

2.2.1 Overview

A custom PCB was designed to integrate all control electronics for the robot. The board serves as the central interface between the STM32 microcontroller and all motor drivers, sensors, and power systems.

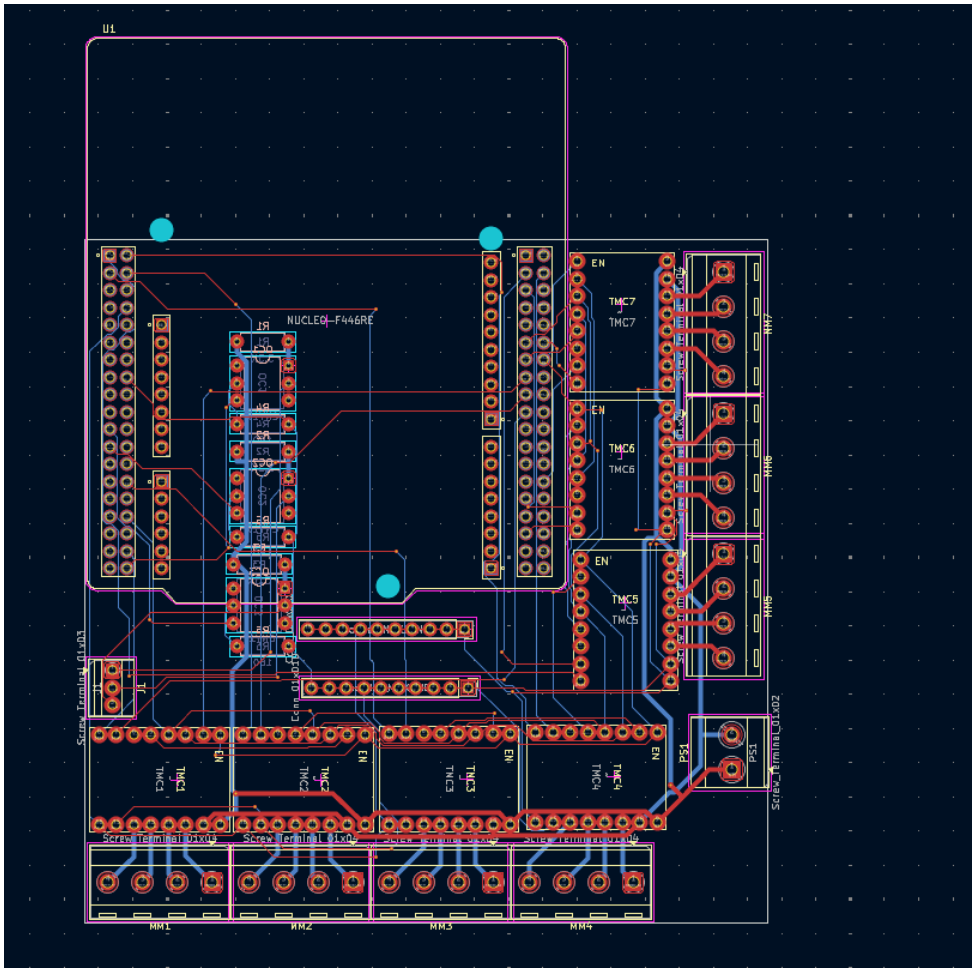

Figure 2.1: Control Board PCB Layout

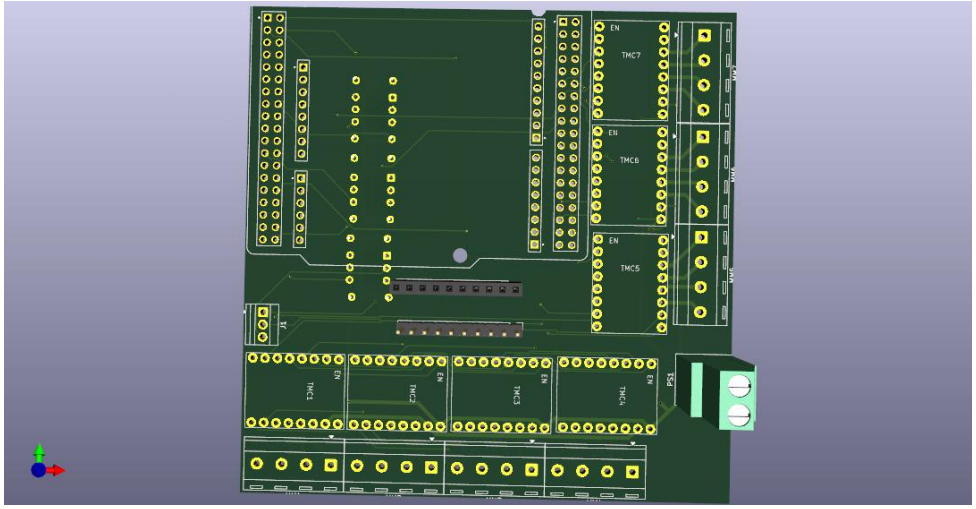

Figure 2.2: Control Board 3D View

Key Features

- **MCU Interface**: Socket for STM32 Nucleo development board
- **Motor Drivers**: 7× TMC5160 BigTreeTech driver slots (6 robot joints + 1 gripper)
- **Motor Outputs**: Connectors for all stepper motors
- **Limit Switches**: Input connectors for all joint limit switches
- **Power Interface**: Motor power input and distribution

2.2.2 Microcontroller

The control board uses an **STM32 Nucleo** development board as the main processor:

- Plugs into dedicated socket on the control board
- Provides all GPIO, timer, and communication interfaces
- USB connection for host communication
- Firmware development and debugging via ST-Link

2.2.3 Motor Drivers

The board accommodates 7 **TMC5160 BigTreeTech** stepper drivers:

Slot	Function	Motor
1	Joint 1	Base rotation
2	Joint 2	Shoulder
3	Joint 3	Elbow
4	Joint 4	Wrist 1
5	Joint 5	Wrist 2
6	Joint 6	Wrist 3
7	Gripper	End effector

Table 2.5: Motor Driver Assignments

2.2.4 Power System

- **Motor Power**: Direct connection for motor supply voltage
- **Logic Power**: Regulated from motor supply or USB

2.2.5 Connectors

- **Motor outputs**: 7× stepper motor connectors (4-pin)
- **Limit switches**: Input connectors for each joint
- **Encoder inputs**: For AS5600 position feedback
- **Power input**: Main power connector

2.3 Electrical Connections

This section presents the schematic diagrams for the main electrical subsystems.

2.3.1 MCU Connections

The STM32 Nucleo board interfaces with all other subsystems:

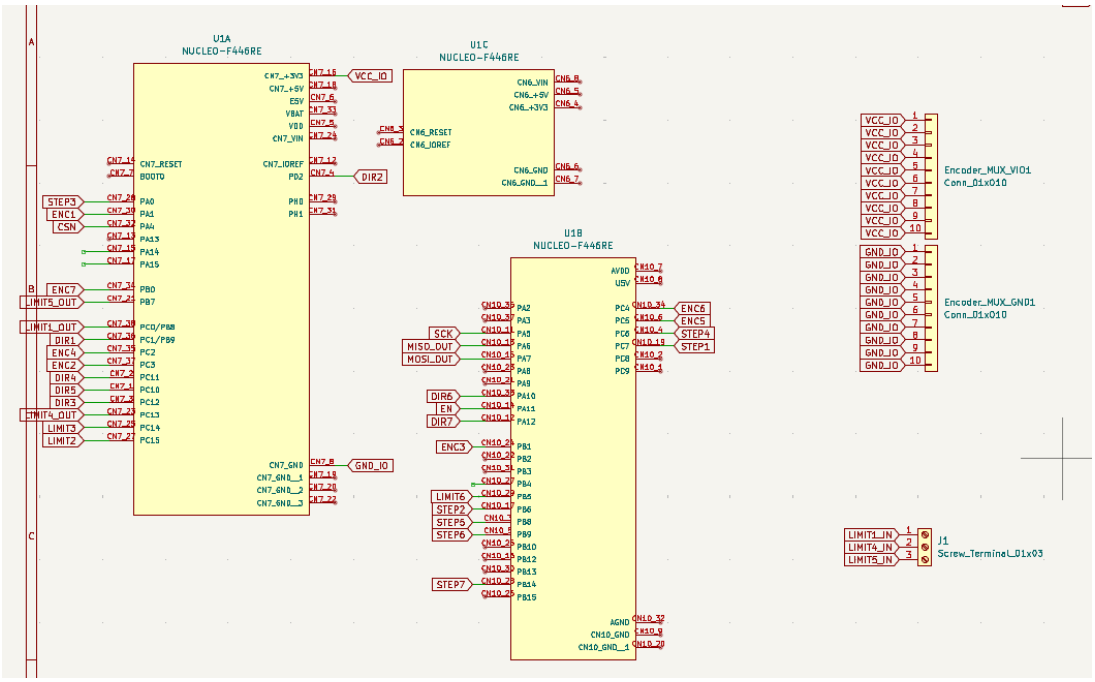

Figure 2.3: MCU Schematic - STM32 Nucleo Connections

2.3.2 Motor Driver Wiring

The TMC5160 motor drivers are connected via **SPI daisy chain** configuration. This allows all 7 drivers to share a single SPI bus, reducing the number of required GPIO pins.

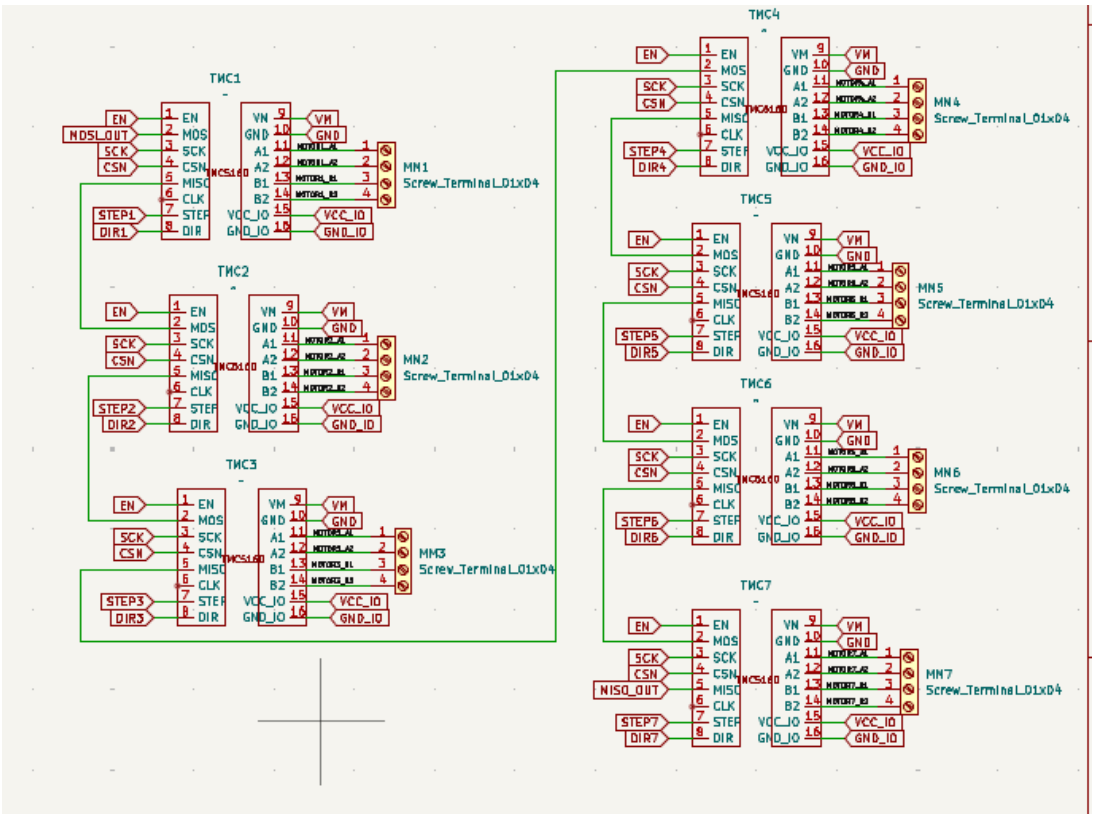

Figure 2.4: Motor Driver Schematic - SPI Daisy Chain Configuration

SPI Daisy Chain

In daisy chain mode:

- All drivers share common SCK (clock) and CS (chip select) lines
- Data passes from SDO of one driver to SDI of the next
- Single SPI transaction addresses all 7 drivers sequentially
- Reduces wiring complexity and MCU pin usage

2.3.3 Limit Switch Wiring

Each joint has a limit switch for homing and end-of-travel protection:

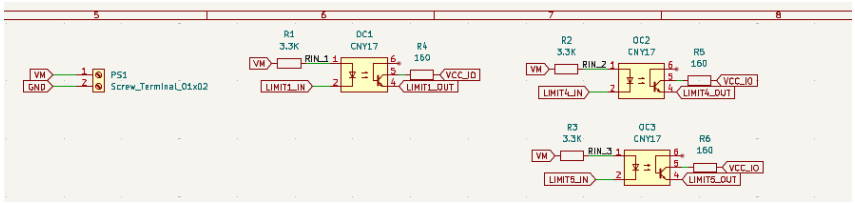

Figure 2.5: Limit Switch Schematic

Chapter 3

MCU Firmware

This chapter describes the firmware running on the STM32 Nucleo microcontroller.

3.1 Firmware Overview

The MCU firmware is responsible for real-time motor control, sensor reading, and communication with the host. It uses FreeRTOS for task management.

3.1.1 System Architecture

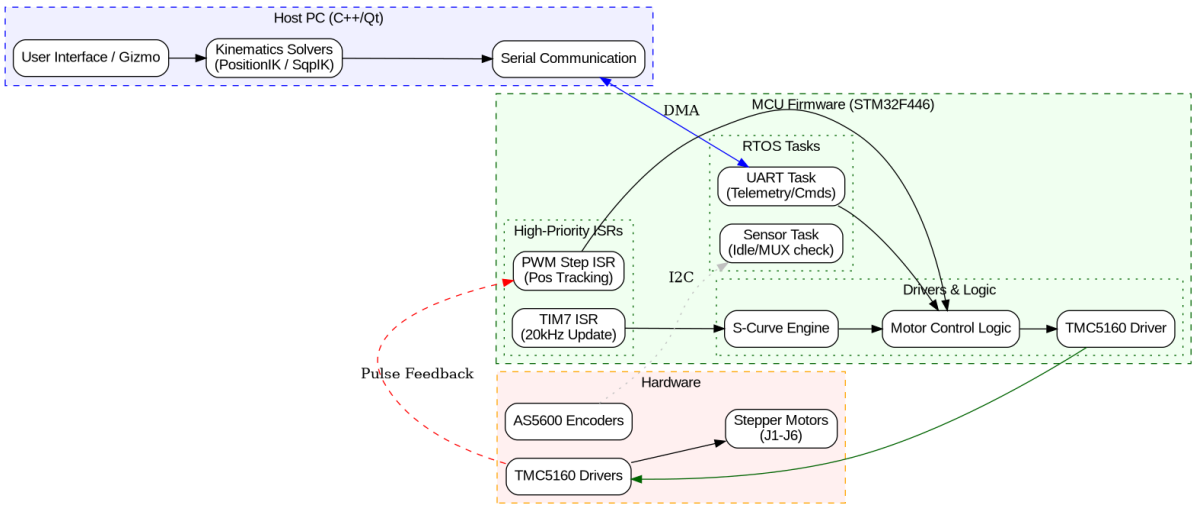

Figure 3.1: System Architecture and Data Flow across Host PC, MCU, and Hardware

3.1.2 RTOS Tasks

The firmware utilizes a hybrid architecture combining high-frequency interrupts for motion control with RTOS tasks for communication and low-priority monitoring:

Component	Priority/Rate	Function
TIM7 ISR	20 kHz	S-Curve engine update, PWM frequency modulation
PWM Step ISR	Event-based	Real-time position tracking, target-reached stopping
UART Task	High	DMA-based telemetry streaming and command parsing
Sensor Task	Low	PCA9548A MUX detection and health monitoring

Table 3.1: Real-Time Software Components

3.2 Motor Control

3.2.1 TMC5160 Driver Interface

The firmware communicates with TMC5160 drivers via SPI:

- Daisy chain configuration for all 7 drivers
- Register access for current, velocity, and position control
- StallGuard feedback for sensorless homing

3.2.2 Step Generation

Motor steps are generated using hardware timers:

- Timer-based step pulse generation
- Velocity and acceleration limiting
- Smooth start/stop transitions

The firmware implements a high-performance 7-segment S-Curve motion profile to ensure continuous acceleration and minimize mechanical vibrations. Unlike standard trapezoidal profiles, the S-Curve limits the third derivative of position (jerk), preventing the "infinite jerk" spikes that lead to motor skipping and structural resonance.

3.2.3 Mathematical Foundation

The motion profile is governed by the constant jerk J, which is the rate of change of acceleration a. The kinematic states are derived through successive integration:

$$a(t) = a_0 + \int_0^t J(\tau)d\tau \tag{3.1}$$

$$v(t) = v_0 + \int_0^t a(\tau)d\tau = v_0 + a_0 t + \frac{1}{2}Jt^2 \tag{3.2}$$

$$s(t) = s_0 + \int_0^t v(\tau)d\tau = s_0 + v_0 t + \frac{1}{2}a_0 t^2 + \frac{1}{6}Jt^3 \tag{3.3}$$

3.2.4 The 7-Segment Profile

A full point-to-point motion consists of seven distinct phases:

1. **Jerk-up**: Jerk is constant ($+J_{max}$), acceleration increases linearly.
2. **Constant Accel**: Jerk is zero, acceleration is constant (a_{max}).
3. **Jerk-down**: Jerk is constant ($-J_{max}$), acceleration decreases to zero as velocity reaches the maximum setpoint.
4. **Constant Velocity**: Jerk and acceleration are zero.
5. **Jerk-down**: Jerk is constant ($-J_{max}$), acceleration linearly decreases (deceleration increases).
6. **Constant Decel**: Jerk is zero, deceleration is constant ($-a_{max}$).
7. **Jerk-up**: Jerk is constant ($+J_{max}$), deceleration returns to zero as the target position is reached.

3.2.5 Implementation Strategy

The firmware executes this engine within the **TIM7 ISR at 20 kHz**. This high frequency (50 μs ticks) allows for quasi-continuous updates of the output PWM frequency, ensuring the motor follows the ideal curve with negligible quantization error.

Adaptive Deceleration

Because the MCU must handle dynamic target updates and external interruptions, the engine does not simply "blindly" run a pre-calculated curve. Instead, it performs a **deceleration check every 1 ms**:

- It continuously calculates the *stopping distance* required to reach zero velocity from the current speed using maximum jerk/decel limits.
- When the *remaining distance* to the target is less than or equal to the stopping distance, it triggers the transition from constant velocity to the deceleration phases (Phase 5).
- **Overshoot Correction**: In the final pulses (Phase 7), any remaining floating-point error is compensated by an adaptive "dist-remove-per-cycle" calculation, guaranteeing that the final pulse lands exactly on the integer step target.

3.3 Communication Protocol

Communication with the host software is implemented over a high-speed USB-CDC serial link (baud rate 921,600). The protocol uses a robust binary framing to ensure data integrity and low latency.

3.3.1 Framing Structure

All packets, regardless of direction, share a common header and footer structure:

- **Start Frame**: `0xAA 0x55` (2 bytes)
- **Length**: Payload size (2 bytes, Little-Endian)
- **Payload**: Actual data (variable length)
- **Checksum**: XOR sum of all payload bytes (1 byte)

3.3.2 Outgoing Format (MCU → PC)

The MCU streams telemetry at 100 Hz. The primary telemetry packet (Type `0x01`) payload consists of:

- **Joint Data**: 6 joints × (Position, Velocity, Acceleration) as 32-bit floats (72 bytes total)
- **Robot State**: 1 byte integer (Current operational mode)

3.3.3 Incoming Format (PC → MCU)

The host sends control and motion commands. The payload structure varies by command type:

- **State Commands**: 1 byte (e.g., CMD_ESTOP, CMD_HOME)
- **Motion Commands**: 2 bytes (Command Type + Action) followed by 6 joints × (Position, Velocity) as 32-bit floats (50 bytes total)

Chapter 4

Motion Control Software and Kinematics

This chapter describes the kinematics theory and the host-side software for computation and control.

4.1 Kinematics Theory

Kinematics describes the motion of the robot without considering the forces that cause it. For our 6-DOF arm, we define the relationship between joint space $\mathbf{q} \in \mathbb{R}^6$ and task space $\mathbf{x} \in SE(3)$.

4.1.1 Forward Kinematics

Forward Kinematics (FK) is the mathematical process of determining the pose (position and orientation) of the end-effector given a set of joint angles. For a 6-DOF manipulator like our arm, this involves mapping a vector of joint angles $\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6]^T$ to a homogeneous transformation matrix ${}^0T_6 \in SE(3)$.

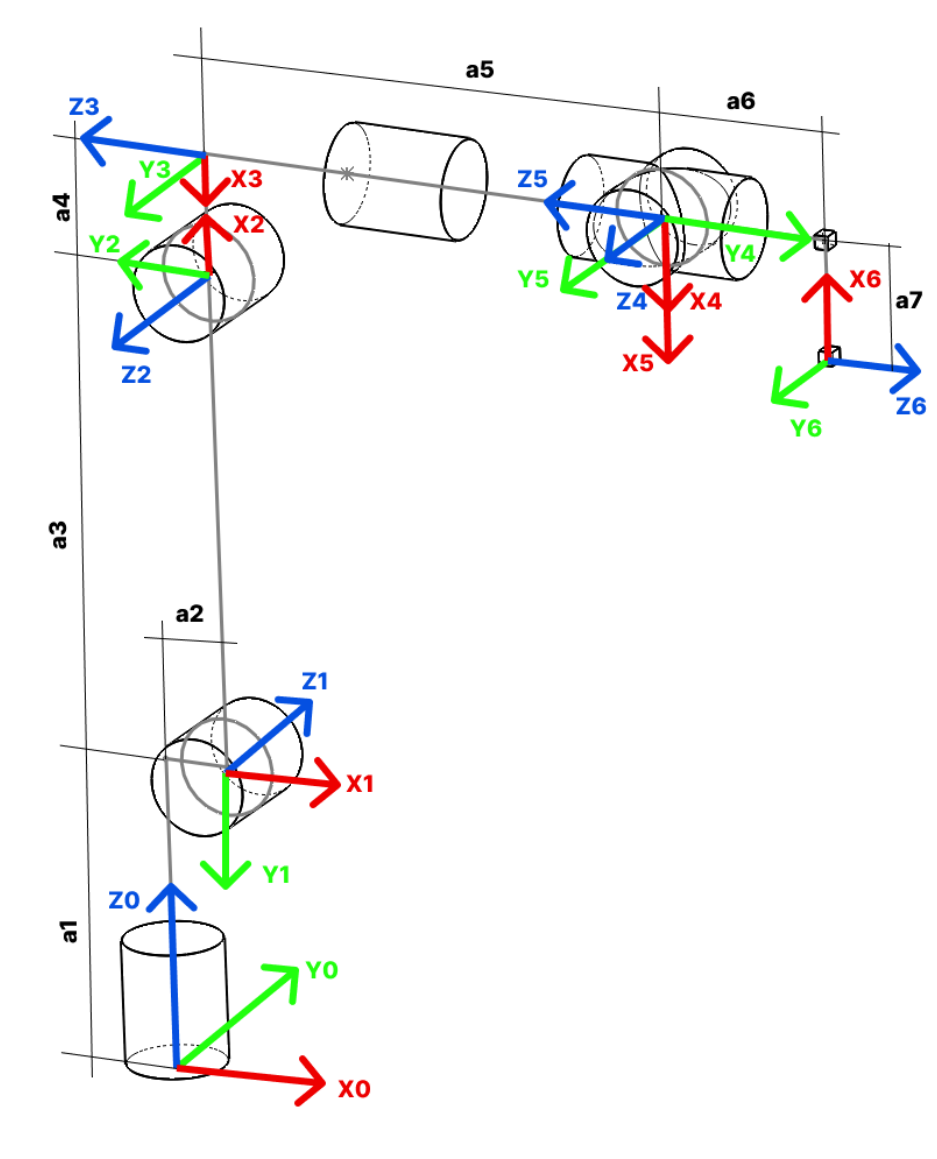

Figure 4.1: Robot Kinematic Diagram showing joint axes and coordinate frames

The Denavit-Hartenberg (DH) Convention

To systematically represent the relationship between adjacent links, we adopt the **Standard Denavit-Hartenberg (DH)** convention. This method assigns a coordinate frame to each joint and describes the transformation from frame $i-1$ to frame i using four parameters:

- **Joint Angle** (θ_i): Rotation about the z_{i-1} axis. This is the variable component for revolute joints.
- **Link Offset** (d_i): Distance along the z_{i-1} axis from the origin of frame $i-1$ to the common normal between z_{i-1} and z_i.
- **Link Length** (a_i): Distance along the x_i axis (the common normal) between the z_{i-1} and z_i axes.
- **Link Twist** (α_i): Rotation about the x_i axis required to align z_{i-1} with z_i.

The individual transformation matrix ${}^{i-1}T_i$ is the result of four successive operations: $Rot_z(\theta_i) \rightarrow Trans_z(d_i) \rightarrow Trans_x(a_i) \rightarrow Rot_x(\alpha_i)$.

$$
{}^{i-1}T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.1}
$$

Robot DH Parameters

Based on the physical dimensions of our robot arm, the following DH parameters define the kinematic chain. The base height H_1, link lengths L_2, L_3, and wrist offsets W_1, W_2 are derived from the URDF model.

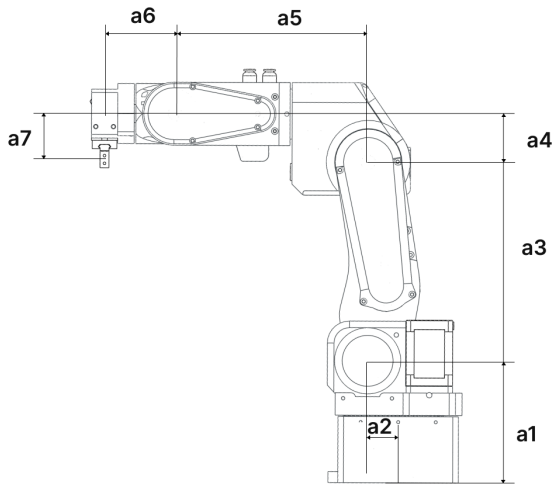

Figure 4.2: Physical Link Lengths

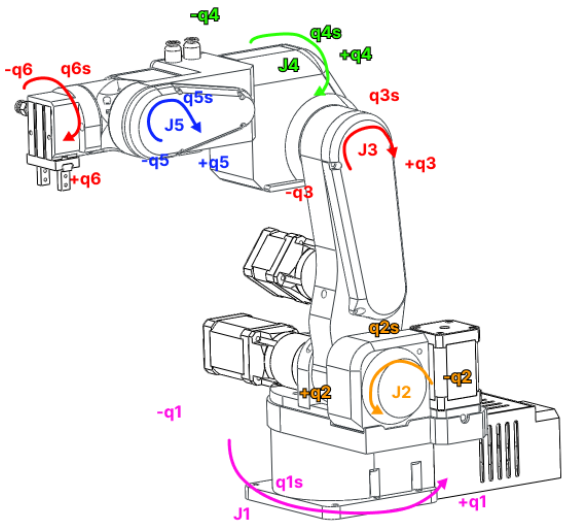

Figure 4.3: Joint Rotation Directions

Link	θ_i	d_i	a_i	α_i
1	q_1	110.5 mm	23.4 mm	$-90°$
2	q_2	0 mm	180.0 mm	$180°$
3	q_3	0 mm	43.5 mm	$90°$
4	q_4	176.35 mm	0 mm	$-90°$
5	q_5	0 mm	51.0 mm	$90°$
6	q_6	52.0 mm	0 mm	$0°$

Table 4.1: DH Parameters for the 6-DOF Robot Arm

The total transformation from the base to the end-effector is obtained by multiplying these matrices in sequence:

$$
{}^0T_{ee}(\mathbf{q}) = {}^0T_1(q_1) \cdot {}^1T_2(q_2) \cdot {}^2T_3(q_3) \cdot {}^3T_4(q_4) \cdot {}^4T_5(q_5) \cdot {}^5T_6(q_6) \tag{4.2}
$$

End-Effector Pose Extraction

The resulting 4×4 homogeneous transformation matrix ${}^0T_{ee}$ contains the full pose information relative to the base frame. It is partitioned as follows:

$$ {}^0T_{ee} = \begin{bmatrix} \mathbf{n} & \mathbf{s} & \mathbf{a} & \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.3} $$

where:

- **Position**: The vector $\mathbf{p} = [p_x, p_y, p_z]^T$ represents the Cartesian coordinates of the end-effector origin.
- **Orientation**: The 3×3 submatrix $\mathbf{R} = [\mathbf{n}, \mathbf{s}, \mathbf{a}]$ represents the rotation of the end-effector. The columns $\mathbf{n}$ (normal), $\mathbf{s}$ (sliding), and $\mathbf{a}$ (approach) are unit vectors defining the orientation of the final link's axes in the base frame.

In our software, this position vector and rotation matrix are used directly for error computation in the numerical solvers and for visualization in the 3D interface.

4.1.2 The Jacobian Matrix

The Jacobian matrix $\mathbf{J}(\mathbf{q})$ is a fundamental operator in robotics that maps velocities from joint space to task space. For our 6-DOF manipulator, it relates the joint velocity vector $\dot{\mathbf{q}} \in \mathbb{R}^6$ to the end-effector twist $\xi = [\mathbf{v}, \omega]^T \in \mathbb{R}^6$:

$$ \begin{bmatrix} \mathbf{v} \\ \omega \end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{4.4} $$

where $\mathbf{v}$ is the linear velocity and ω is the angular velocity of the end-effector.

Geometric Jacobian Derivation

The Jacobian is composed of six columns, $\mathbf{J} = [\mathbf{j}_1, \mathbf{j}_2, \ldots, \mathbf{j}_6]$, where each column represents the contribution of a single joint to the end-effector velocity. Since all joints in our robot are revolute, the i-th column is derived geometrically as:

$$ \mathbf{j}_i = \begin{bmatrix} \mathbf{z}_{i-1} \times (\mathbf{p}_{ee} - \mathbf{p}_{i-1}) \\ \mathbf{z}_{i-1} \end{bmatrix} \tag{4.5} $$

where:

- $\mathbf{z}_{i-1}$ is the unit vector along the axis of rotation of joint i.
- $\mathbf{p}_{ee}$ is the position of the end-effector in the base frame.
- $\mathbf{p}_{i-1}$ is the position of the origin of frame $i-1$ in the base frame.

The Jacobian is highly configuration-dependent. When $\mathbf{J}(\mathbf{q})$ becomes singular (loses rank), the robot loses one or more degrees of freedom in task space. This occurs when joint axes align or the arm reaches its workspace boundaries.

4.1.3 Singularity Handling: Selectively Damped Least Squares

To maintain stability near singularities, we utilize the **Selectively Damped Least Squares (SDLS)** method. Unlike standard Damped Least Squares (Tikhonov regularization) which adds a uniform penalty to all joints, SDLS specifically targets only the singular directions in the task space.

Through Singular Value Decomposition (SVD), the Jacobian is decomposed as:

$$\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{4.6}$$

where the columns of $\mathbf{V}$ are joint-space vectors. We construct a damping matrix $\mathbf{\Lambda}$ that acts only in the basis of these vectors:

$$\mathbf{\Lambda} = \sum_{i=1}^{n} \lambda_i^2 \mathbf{v}_i \mathbf{v}_i^T \tag{4.7}$$

The per-mode damping factor λ_i is computed based on the singular value σ_i:

$$\lambda_i = \begin{cases} \lambda_{max}\left(1 - \frac{\sigma_i}{\sigma_{threshold}}\right)^2 & \text{if } \sigma_i < \sigma_{threshold} \\ 0 & \text{otherwise} \end{cases} \tag{4.8}$$

This approach ensures that "healthy" directions remain precise while "singular" directions are smoothly constrained, preventing infinite joint speeds and numerical instability.

4.1.4 Inverse Kinematics

Inverse Kinematics (IK) is the process of finding the joint angles $\mathbf{q}$ that result in a desired end-effector pose. Unlike FK, IK is non-linear and can have multiple solutions, or no solution if the target is outside the workspace.

Analytical Inverse Kinematics

Analytical (closed-form) Inverse Kinematics is the process of finding joint angles $\mathbf{q}$ algebraically. Our robot's architecture allows for a **kinematic decoupling** strategy, which divides the 6-DOF problem into two smaller, independent subproblems: a 3-DOF positional problem and a 3-DOF orientational problem.

1. Kinematic Decoupling (Wrist Center) The key requirement for decoupling is that the axes of the last three joints (Joints 4, 5, and 6) intersect at a single point, known as the **wrist center** (P_w). Because rotational motion of the wrist does not move P_w, its position is determined solely by the first three joints.

Given a target end-effector position $\mathbf{p}_{target}$ and its approach vector $\mathbf{a}$ (the z-axis of the end-effector frame), the wrist center is calculated as:

$$\mathbf{p}_w = \mathbf{p}_{target} - d_6 \cdot \mathbf{a} \tag{4.9}$$

where $d_6 = 52.0\,\text{mm}$ is the distance from the wrist center to the end-effector tip.

2. Solving for the Arm (Joints 1, 2, 3) With $\mathbf{p}_w = [x_w, y_w, z_w]^T$ known in the base coordinate frame:

- **Base Rotation (q_1)**: The first joint rotates the entire arm to align with the $x - y$ projection of the wrist center:

$$q_1 = \text{atan2}(y_w, x_w) \quad (\text{or } q_1 + \pi \text{ for reversed base}) \tag{4.10}$$

- **Planar Geometry (q_2, q_3)**: Once q_1 is fixed, the problem simplifies to a 2D planar linkage. We solve for q_2 and q_3 using the **Law of Cosines** based on the triangle formed by L_2, L_3, and the radial distance to P_w. This step yields two solutions per base configuration: *Elbow Up* and *Elbow Down*.

3. Solving for the Wrist (Joints 4, 5, 6) After calculating the arm joints, the rotation matrix from the base to Joint 3, ${}^0R_3$, is fully defined. To reach the target orientation $\mathbf{R}_{target}$, the wrist must provide the remaining rotation ${}^3R_6$:

$${}^3R_6 = ({}^0R_3)^T \cdot \mathbf{R}_{target} \tag{4.11}$$

The matrix ${}^3R_6$ corresponds to a standard **Z-Y-Z Euler sequence** defined by the wrist joints q_4, q_5, q_6. Equating the elements of ${}^3R_6$ to the Euler rotation matrix allows us to extract the angles:

$$q_5 = \pm \,\text{acos}(R_{33}) \quad (\text{where } R_{33} \text{ is the bottom-right element}) \tag{4.12}$$

The choice of sign for q_5 defines the **Wrist Flip** (normalized vs. flipped configuration).

4. Solution Selection and Configuration Ranking For any reachable target, the analytical solver [6] generates a candidate set of up to **8 valid solutions** (2 Base $\times$ 2 Elbow $\times$ 2 Wrist). Because analytical IK can be discontinuous—exhibiting "jerk" between branches—our solver utilizes a ranking and blending system to select the most stable solution:

- **Configuration Hysteresis**: A large "Branch Penalty" is applied to solutions that require a change in the robot's fundamental mode (e.g., flipping the base or elbow). This ensures the robot stays within its current kinematic branch as long as it is reachable.

- **Singularity-Aware Metric**: Standard Euclidean distance is insufficient near singularities. Our metric downweights Joint 4 and 6 when near a wrist singularity ($|\sin(q_5)| \approx 0$), as they become redundant. We also penalize deviations in the joint sum $q_4 + q_6$ to maintain consistent orientation.

- **Dynamic Blend Zones**: We implement a *Lock-and-Blend* strategy. Below a 3° threshold, Joint 4 and 6 are hard-locked to their previous values. Between 3° and 8°, a linear blend factor smoothly transitions the solution from the previous state to the new analytical goal.

- **Manipulability Ranking**: Solutions are ranked using the *Yoshikawa Manipulability Index* ($\sqrt{\det(JJ^T)}$) [4]. Configurations that are further away from geometric singularities (vertical arm, fully extended elbow) are prioritized.

- **Velocity-Aware Penalty**: The solver predicts the required joint velocity ($\dot{q} = \Delta q / \Delta t$). Solutions that would require velocities exceeding the motor's physical limits are heavily penalized to avoid physical tripping.

4.1.5 Role in the Control Loop

It is important to note that this **Analytical Inverse Kinematics** solver [6] is primarily used for **simulation and global initialization**. Because it is solved algebraically at a discrete pose, it is not inherently continuous. In the real-time control system, it serves as a **Seeder** for the **Differential QP Controller** (described in Section 4.3). The analytical solver provides a globally valid starting point, while the QP-based Jacobian solver ensures perfectly smooth, continuous joint trajectories during execution.

4.2 Differential IK via Quadratic Programming: A Constraint-Aware Framework

Executive Summary

This report details our **Differential QP Controller**, which hybridizes analytical seeding with a numerical Quadratic Programming (QP) solver. By integrating **Selectively Damped Least Squares (SDLS)** and strict physical constraints via OSQP, the controller ensures robust, singularity-aware motion that overcomes the limitations of traditional numerical methods.

4.2.1 Introduction and Problem Formulation

The Kinematic Control Challenge in Modern Robotics

Inverse Kinematics (IK) constitutes the foundational layer of robotic manipulation, serving as the critical mathematical bridge between high-level task planning in Cartesian space and the low-level joint servo control required by actuators. The problem is formally defined as finding a joint configuration vector $\mathbf{q} \in \mathbb{R}^n$ that satisfies the non-linear mapping $\mathbf{x}_{des} = f(\mathbf{q})$, where $\mathbf{x}_{des} \in SE(3)$ represents the desired end-effector pose and $f : \mathbb{R}^n \to SE(3)$ is the forward kinematics function.

While this formulation appears straightforward, practical implementation in industrial and collaborative environments introduces significant complexity. The function $f(\mathbf{q})$ is highly non-linear, involving trigonometric compositions that result in non-convex solution spaces. Redundant manipulators possess an infinite set of solutions known as the self-motion manifold. Conversely, non-redundant manipulators may have multiple discrete geometric solutions (branches), such as "elbow-up" or "elbow-down" configurations.

Limitations of Classical Numerical Solvers

Historically, the robotics community has relied on analytical and numerical solvers. Analytical solvers are fast and complete but specific to geometry and cannot easily incorporate inequality constraints. Numerical iterative methods offer generality by solving the problem at the velocity level:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{4.13}$$

where $\mathbf{J}(\mathbf{q}) = \partial f / \partial \mathbf{q} \in \mathbb{R}^{m \times n}$ is the manipulator Jacobian. However, these classical numerical approaches suffer from three critical pathologies:

- **Local Minima Entrapment**: Gradient-based solvers descend the error landscape from an initial guess. If initialized in the basin of attraction of a local minimum, the

solver may converge to a state where the Cartesian error is non-zero, yet no local movement can reduce it.

- **Singularity Instability**: Near kinematic singularities (where $\text{rank}(\mathbf{J}) < m$), the Jacobian becomes ill-conditioned. Pseudoinverse-based controllers command infinite joint velocities, leading to dangerous "jerk" or controller divergence.
- **Constraint Violation**: Pure kinematic inversion ($\dot{\mathbf{q}} = \mathbf{J}^{\dagger}\dot{\mathbf{x}}$) is fundamentally unconstrained. It does not natively ensure that the resulting joint coordinates $\mathbf{q}$ remain within physical limits.

The Optimization Paradigm: Differential QP

Our controller addresses these limitations by reformulating the IK problem as a **Constrained Quadratic Program (QP)**. By expressing the control objective as the minimization of a quadratic cost function subject to linear inequality constraints, the controller can optimally trade off tracking error against control effort while rigorously respecting physical boundaries.

This implementation utilizes the **OSQP (Operator Splitting Quadratic Program)** solver, a numerical engine tailored for sparse, convex problems. Crucially, the controller augments this optimization core with a **Hybrid Architecture**, utilizing an analytical solver [6] to seed the optimization. This combines the global topology awareness of analytical methods with the local constraint-handling capability of numerical optimization.

4.2.2 Mathematical Framework and Derivations

The efficacy of the controller rests on a rigorous mathematical formulation that translates physical robotic constraints into the language of convex optimization.

Differential Kinematics and Error Dynamics

The controller operates in a closed-loop real-time cycle (1 kHz). At each timestep t, the goal is to compute a joint velocity vector $\dot{\mathbf{q}}$ that minimizes the error between the current pose and the target pose.

Task-Space Error Formulation The translational error $\mathbf{e}_p$ is the linear difference: $\mathbf{e}_p = \mathbf{p}_{target} - \mathbf{p}_{current}$. The rotational error $\mathbf{e}_o$ utilizes unit quaternions $\mathbf{Q} = [w, x, y, z]^T$. To handle the double cover of $SO(3)$, we explicitly check the inner product:

$$\text{if } (\mathbf{Q}_{curr} \cdot \mathbf{Q}_{target} < 0) \implies \mathbf{Q}_{target} \leftarrow -\mathbf{Q}_{target} \tag{4.14}$$

The orientation error is computed as: $\mathbf{e}_o = 2\ln(\mathbf{Q}_{target} \otimes \mathbf{Q}_{curr}^{-1})$. This prevents "unwinding" jumps and ensures rotation along the geodesic path.

Reference Velocity Generation The controller employs a Proportional (P) control law in task space:

$$\mathbf{v}_{ref} = K_p \begin{bmatrix} \mathbf{e}_p \\ \mathbf{e}_o \end{bmatrix} \tag{4.15}$$

This linearizes the tracking dynamics, treating the robot as a first-order kinematic system.

4.2.3 Differential QP-Based Inverse Kinematics (OSQP Formulation)

1. Optimization problem being solved

At every control cycle, the solver computes a joint velocity command $\dot{\mathbf{q}} \in \mathbb{R}^n$ that best tracks a desired Cartesian velocity while respecting joint limits. The problem is formulated as the following convex quadratic program:

$$\min_{\dot{\mathbf{q}}} \quad \frac{1}{2}||\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} - \mathbf{v}_{ref}||^2_{\mathbf{W}} + \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{\Lambda}\dot{\mathbf{q}} \tag{4.16}$$

$$\text{s.t.} \quad \dot{\mathbf{q}}_{min} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{max} \tag{4.17}$$

Where:

- $\mathbf{J} \in \mathbb{R}^{6\times n}$ is the geometric Jacobian.
- $\mathbf{v}_{ref} = K_p\mathbf{e}$ is the desired Cartesian velocity.
- $\mathbf{W}$ is a task-space weighting matrix.
- $\mathbf{\Lambda}$ is an adaptive SDLS damping matrix.
- Constraints enforce joint velocity and joint-limit damping bounds.

2. Mapping to standard QP form (OSQP)

OSQP solves problems of the form:

$$\min_{x} \quad \frac{1}{2}x^TPx + q^Tx \tag{4.18}$$

$$\text{s.t.} \quad l \leq Ax \leq u \tag{4.19}$$

where $x \in \mathbf{R}^n$ is the optimization variable and $P \in \mathbf{S}^n_+$ a positive semidefinite matrix [5].

3. Objective function mapping

3.1 Quadratic term (Hessian) Expanding the least-squares objective:

$$||\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}_{ref}||^2_{\mathbf{W}} = \dot{\mathbf{q}}^T\mathbf{J}^T\mathbf{W}\mathbf{J}\dot{\mathbf{q}} - 2\mathbf{v}^T_{ref}\mathbf{W}\mathbf{J}\dot{\mathbf{q}} + \text{const} \tag{4.20}$$

Thus, the Hessian is defined as: $\mathbf{P} = \mathbf{J}^T\mathbf{W}\mathbf{J} + \mathbf{\Lambda}$. In practice, a small $\epsilon\mathbf{I}$ is added for numerical positive definiteness.

- **Code mapping**:

```
P_dense = J.transpose() * W_diag * J + Lambda;
P_dense += 1e-6 * Eigen::MatrixXd::Identity(num_joints_,
   num_joints_);
```

3.2 Linear term (gradient) The linear term of the QP is derived as: $\mathbf{q} = -\mathbf{J}^T\mathbf{W}\mathbf{v}_{ref}$.

- **Code mapping**:

```
q_grad_ = -J.transpose() * W_diag * v_ref;
```

4. Constraints mapping

4.1 Constraint matrix Only box constraints are enforced: $\dot{\mathbf{q}}_{min} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{max}$. This is implemented via an identity constraint matrix $\mathbf{A} = \mathbf{I}_{n\times n}$.

- **Code mapping**:

```
A_sparse_.setIdentity();
```

4.2 Velocity Dampers for Joint Constraints An implementation detail is the use of **Velocity Dampers** (also known as potential field limits or control barrier functions at the velocity level) to enforce position limits. This technique converts order-zero constraints (position) into order-one constraints (velocity), making them compatible with the differential QP formulation.

For a joint i with position limits $[q^i_{min}, q^i_{max}]$ and physical velocity limits $[-v^i_{max}, v^i_{max}]$, the admissible velocity interval $[\dot{q}^i_{lower}, \dot{q}^i_{upper}]$ is dynamically adjusted based on the proximity to the limits.

The upper velocity bound is defined as:

$$\dot{q}^i_{upper}(t) = \min\left(v^i_{max}, \quad \xi\frac{q^i_{max} - q_i(t)}{\Delta t}\right) \tag{4.21}$$

The lower velocity bound is defined as:

$$\dot{q}^i_{lower}(t) = \max\left(-v^i_{max}, \quad \xi\frac{q^i_{min} - q_i(t)}{\Delta t}\right) \tag{4.22}$$

where ξ is the `velocity_damper_gain_` (typically $\xi \in [0.1, 1.0]$).

This formulation acts as a **"soft wall"**:

- **Far from limit**: The term $\xi\frac{d_{limit}}{\Delta t}$ is large, so the bound is simply the physical motor limit v_{max}.
- **Approaching limit**: As the distance to the limit approaches zero, the allowable velocity linearly decreases, forcing the robot to decelerate smoothly.
- **At limit**: The velocity bound becomes zero (or negative), strictly preventing any motion that would violate the limit while permitting motion away from it.

These dynamic bounds are fed directly into the QP constraint vectors l and u (l_{eff} and u_{eff} in the code), ensuring that the optimization result is always **"viable"**—meaning a valid future trajectory exists.

Constraint Type	Physical Meaning	Mathematical Implementation
Box Constraint	Actuator Speed Limits	$\dot{q} \in [-v_{max}, v_{max}]$
Barrier Constraint	Joint Position Limits	$\dot{q} \leq \xi(q_{max} - q)/\Delta t$
Optimization	Task Tracking	$\min \lvert\lvert\mathbf{J}\dot{\mathbf{q}} - \mathbf{v}_{ref}\rvert\rvert^2$

- **Code mapping**:

```
u_eff_(i) = vel_upper;
l_eff_(i) = -vel_lower;
```

5. Selectively Damped Least Squares (SDLS)

To avoid over-damping well-conditioned directions, damping is applied per singular mode [1]:

$$\mathbf{\Lambda} = \sum_i \lambda_i^2 \mathbf{v}_i \mathbf{v}_i^T \tag{4.23}$$

Where the per-mode damping λ_i follows a quadratic ramp:

$$\lambda_i = \begin{cases} \lambda_{max}(1 - \sigma_i/\sigma_{th})^2 & \text{if } \sigma_i < \sigma_{th} \\ 0 & \text{otherwise} \end{cases} \tag{4.24}$$

- **Code mapping**:

```
Lambda += (lambda_i * lambda_i) * V.col(i) * V.col(i).
   transpose();
```

This preserves accuracy in non-singular directions while stabilizing the controller near rank-deficiencies.

6. Final solved QP (summary)

At each control step, OSQP solves the integrated objective:

$$\min_{\dot{\mathbf{q}}} \quad \frac{1}{2}\dot{\mathbf{q}}^T(\mathbf{J}^T\mathbf{W}\mathbf{J} + \mathbf{\Lambda})\dot{\mathbf{q}} - (\mathbf{J}^T\mathbf{W}\mathbf{v}_{ref})^T\dot{\mathbf{q}} \tag{4.25}$$

$$\text{s.t.} \quad \dot{\mathbf{q}}_{min} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{max} \tag{4.26}$$

4.2.4 Hybrid Architecture: Topology and Global Awareness

Behind the optimization core, the controller implements a Motion Lookahead and Hysteresis mechanism to navigate the robot's discrete configuration space.

Predictive Motion Lookahead

Instead of reacting only to the current error, the controller extrapolates the current trajectory to predict the robot's pose in the near future:

$$\mathbf{p}_{lookahead} = \mathbf{p}_{current} + \frac{\mathbf{e}_{pos}}{||\mathbf{e}_{pos}||} \cdot \min(d_{lookahead}, ||\mathbf{e}_{pos}||) \tag{4.27}$$

The analytical IK is solved for this future pose. This effectively asks: "If we continue moving in this direction, will we need to flip the elbow or wrist?" If the `detectBranch` function identifies that the future pose requires a different configuration branch, the controller prepares to switch. This "proto-MPC" behavior allows the robot to anticipate kinematic bifurcations rather than stumbling into them blindly.

Hysteresis-Based Branch Switching

To prevent "chattering"—the rapid, unstable oscillation between configuration branches when near a workspace boundary—the controller implements a hysteresis filter. A switch is only permitted if the cost benefit of switching exceeds a predefined threshold:

$$\text{Cost}_{switch} < \text{Cost}_{stay} - \tau_{hysteresis} \tag{4.28}$$

The cost function `computeBranchCost` penalizes joint distance and manipulability loss:

$$\text{Cost}(q) = \sum (q_i - q_{current,i})^2 + \frac{1}{|\sin(q_{wrist})| + \epsilon} \tag{4.29}$$

This logic ensures that the robot remains in its current valid configuration until the alternative becomes significantly better, mimicking the stability of biological motor control.

4.2.5 Implementation and Performance Evaluation

The system is implemented in C++ using the Eigen library and OSQP. We utilize **Warm-Starting** (initializing with the previous solution) to reduce solver iterations to $\sim 3 - 5$ per cycle, enabling 1 kHz performance.

We evaluate performance through:

- **Singularity Transit**: Verifying stable velocity norms and graceful tracking degradation as $\sigma_{min} \to 0$.
- **Elbow Flip Testing**: Confirming 100% reachability in configuration-switching scenarios where standard local solvers fail.

4.3 Visualization and Control Interface

The robot is operated via a custom graphical user interface developed using the **Qt/QML** framework and C++. This application acts as the high-level controller, bridging the kinematic solvers with the real-time hardware telemetry.

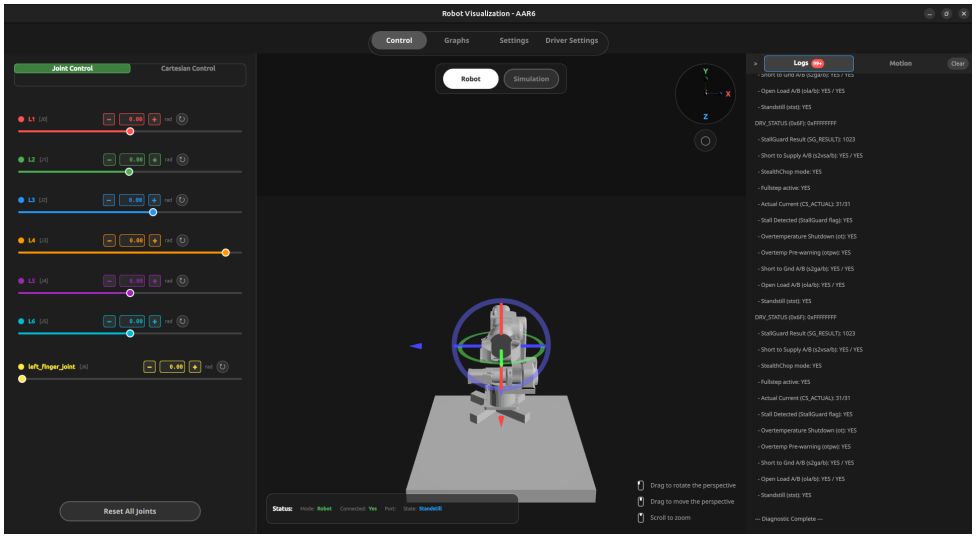

Figure 4.4: The Primary Control Interface showing the 3D robot model and control panels.

4.3.1 Connection and Initialization

The software includes an automated hardware detection system. Upon startup, it scans the available serial ports to identify the custom MCU. Once detected, the system prompts the user to connect:

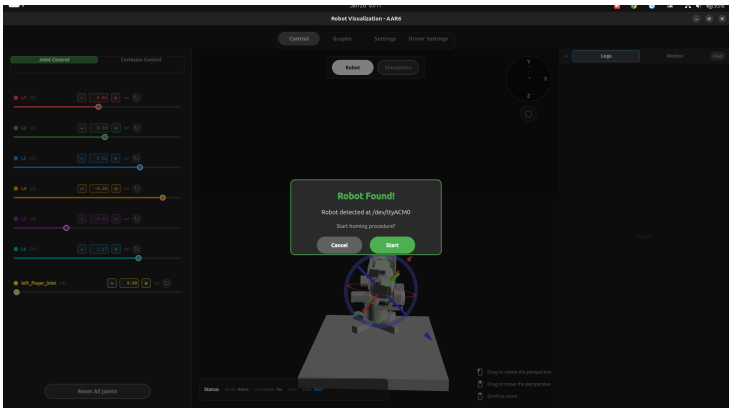

Figure 4.5: The automated connection prompt and MCU detection status.

Upon successful connection, the software initiates a **Homing Sequence**. This ensures that the physical robot's joint positions are synchronized with the internal kinematic model, establishing a verified zero-frame for all subsequent motions.

4.3.2 Control Modes and Input Methods

The interface utilizes a Model-View-Controller (MVC) architecture where the **QML** layer handles the interactive 3D rendering and 2D controls, while the **C++** backend manages the thread-safe communication and IK computation.

Users can control the robot through several input modalities:

1. **Cartesian Drag (3D Gizmo)**: An interactive 3D transform gizmo is attached to the end-effector in the 3D view. Users can drag this gizmo to move the robot in real-time. The software solves the Differential QP-IK at high frequency, streaming smooth trajectories to the MCU.

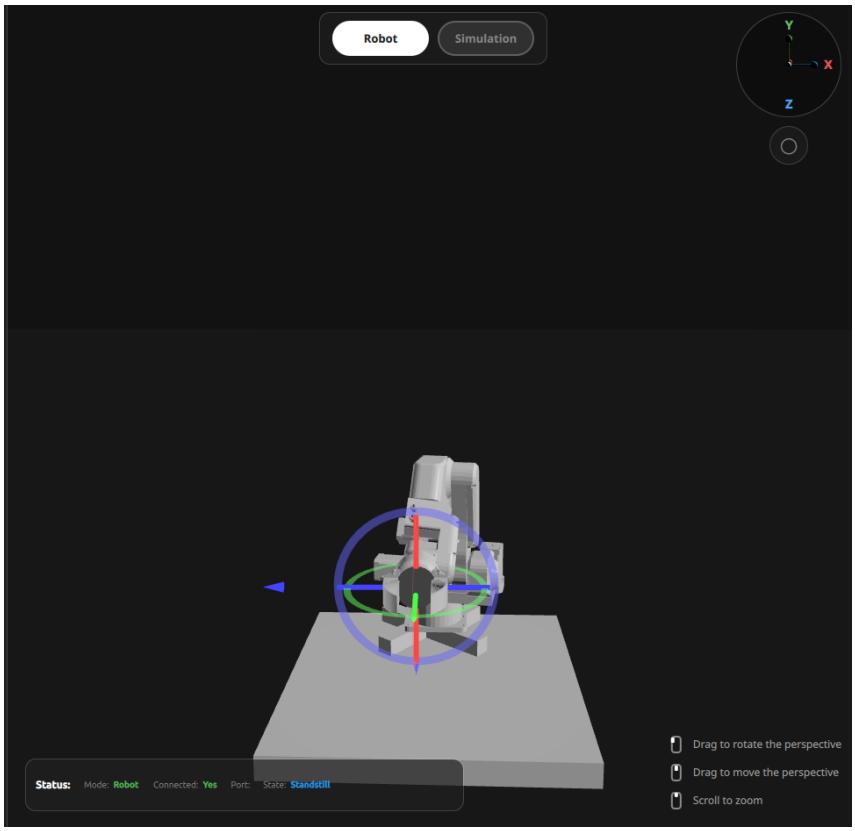

Figure 4.6: Interactive 3D Gizmo for real-time Cartesian manipulation.

2. **Cartesian Command Panel**: For precise numerical input, the Cartesian panel allows users to specify exact target coordinates and orientations. This is particularly useful for tasks requiring sub-millimeter precision that are difficult to achieve through 3D dragging alone.

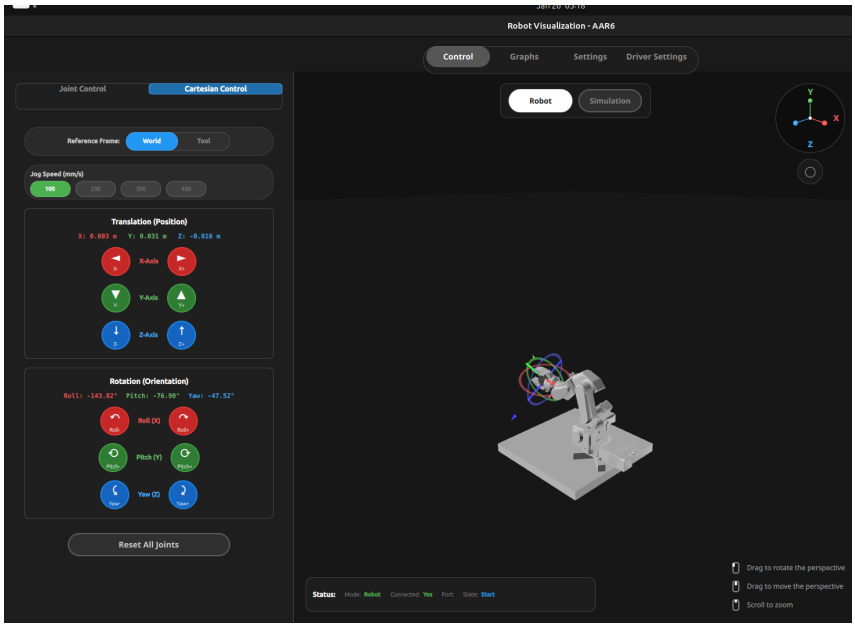

Figure 4.7: Cartesian command and target coordinate panel.

3. **Joint-Space Sliders**: Individual sliders permit direct manipulation of each joint angle (q1-q6). This is useful for manual configuration, diagnostic testing, and verification of individual actuator health.

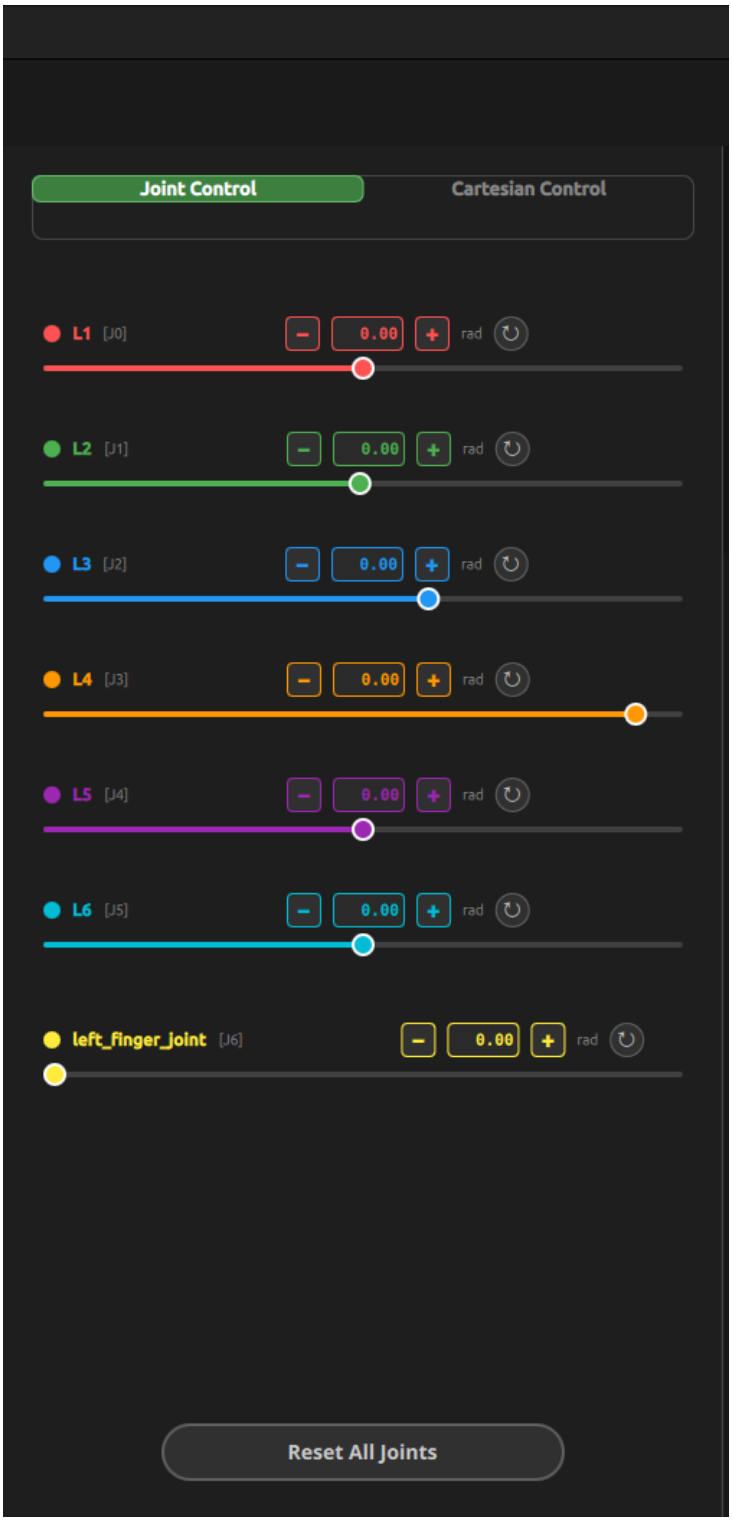

Figure 4.8: Joint-space control interface (Sliders) and Cartesian command panel.

4. **Motion Planning and Waypoints**: The software features a dedicated *Motion Controller* and *Panner*. Users can add specific pose waypoints to a sequence and

play them back. The system interpolates between these points using the internal S-Curve engine to ensure smooth transitions.

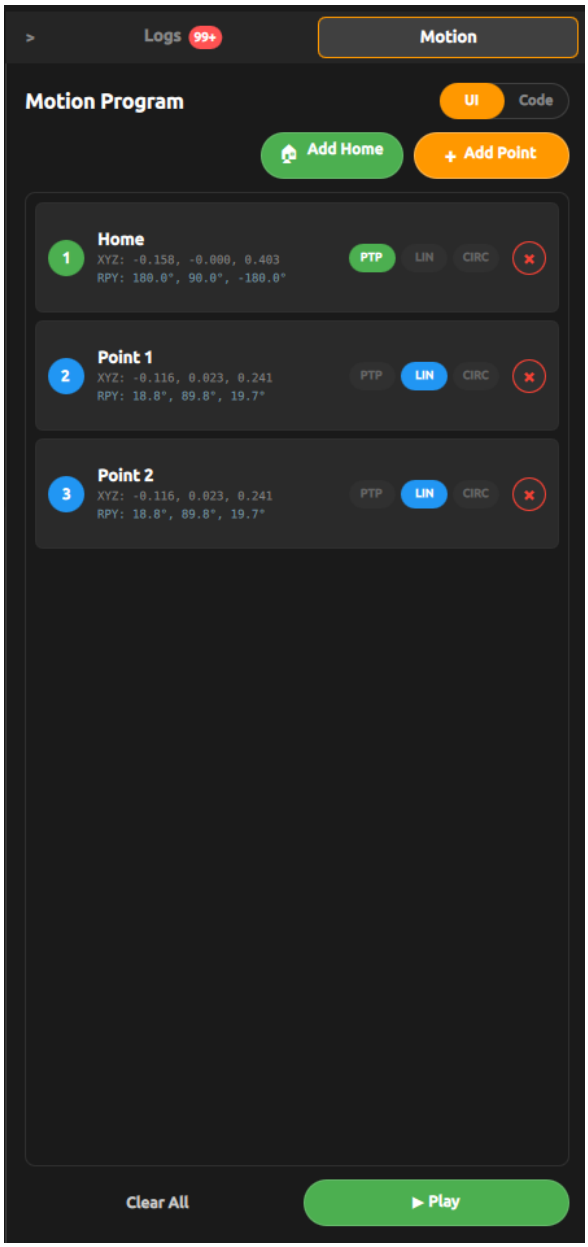

Figure 4.9: Motion waypoint sequence planner.

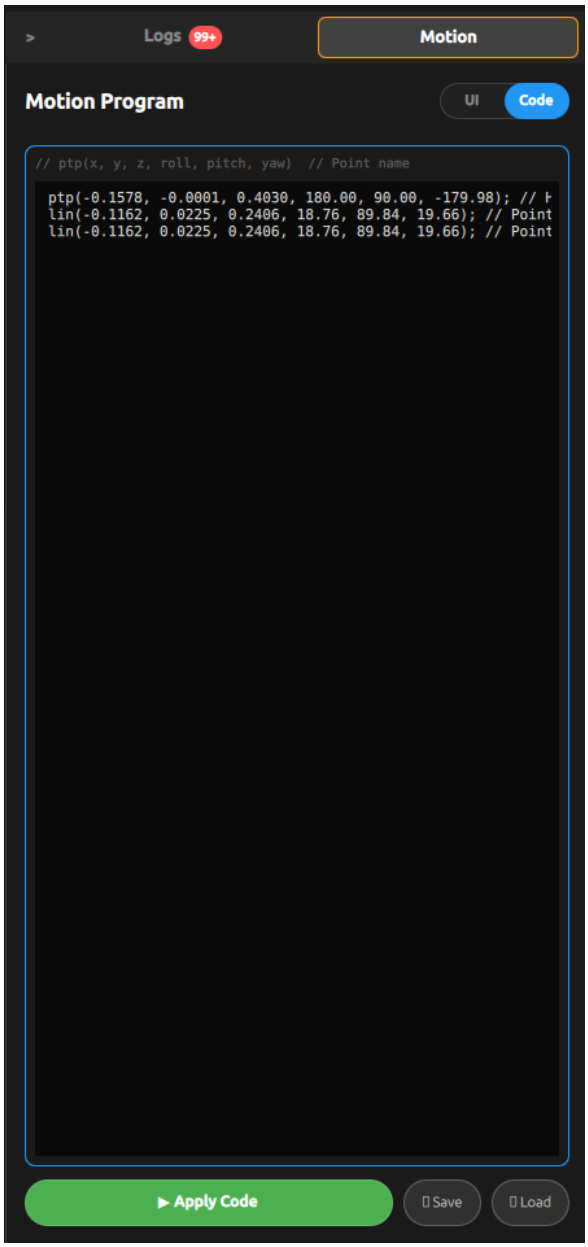

Figure 4.10: Integrated scripting and code interface.

5. **Scripting Interface**: For complex task automation, users can write code directly within the interface to execute logic-based motion sequences, integrating sensor feedback or timed events.

4.3.3 System Telemetry and Visualization

The 3D visualization is a mirror of the physical robot. As joint positions are streamed back from the MCU at 100 Hz, the 3D model is updated via the Forward Kinematics engine, providing the user with an accurate representation of the robot's current state.

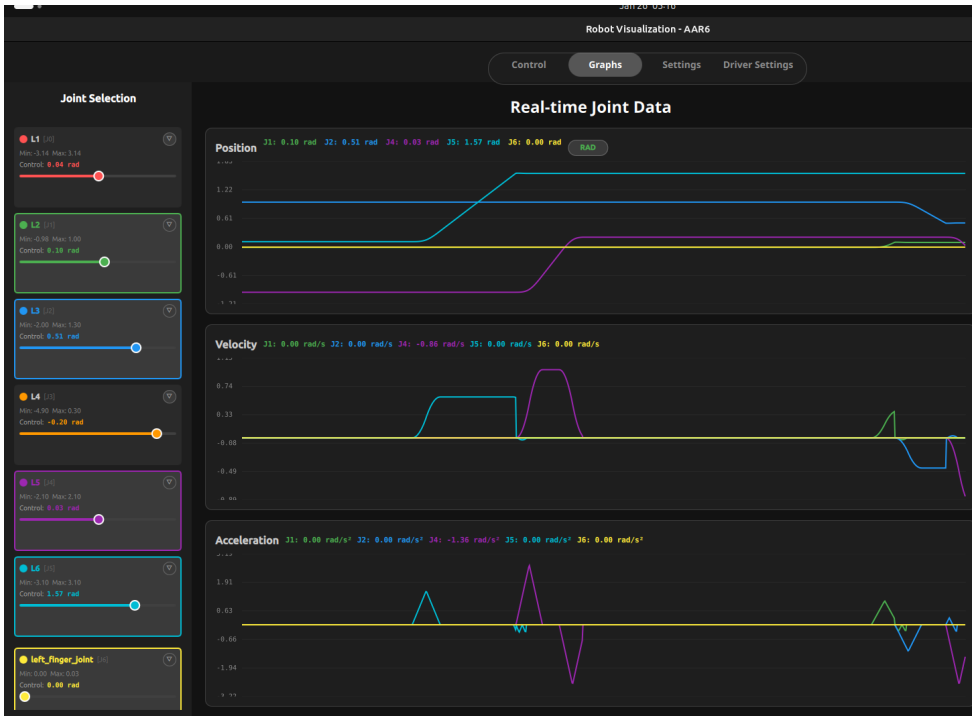

Figure 4.11: Real-time joint velocity and acceleration telemetry graphs.

4.4 Project Status: Limitations and Future Roadmap

The current implementation provides a high-fidelity 6-DOF control system; however, technical constraints across hardware, control logic, and software frameworks define the roadmap for future development.

4.4.1 Hardware and Connectivity

- **Actuator Driver Status (J3)**: The system is currently missing two primary drivers. A temporary "jerry-rigged" driver is currently in use for Joint 3 (J3); however, it is prone to thermal shutdowns due to temperature regulation issues and must be replaced. This failure mode was precipitated by high Back-EMF from J3, which destroyed two previous drivers. A high-quality, high-EMF-rated driver is required for stable long-term operation (TMC5160 Product Page).

- **End-Effector (Gripper) Status**: The gripper is currently disconnected from the control loop. This is due to a combination of time constraints and the aforementioned driver failures, leaving insufficient functional drivers for complete integration.

- **Payload and Mass Distribution**: The NEMA-style stepper motor used for the gripper adds approximately 300 g of dead weight at the end of the arm. This significantly reduces the effective payload capacity and increases the inertial load during high-acceleration maneuvers.

- **Position Encoders (AS5600)**: The AS5600 magnetic encoders are currently not connected to the control loop. Once integrated, these will enable **Admittance**

Control and adjustable arm stiffness. This hardware capability will allow for "lead-through" programming, where the user can teach the robot new paths simply by physically moving the arm.

4.4.2 Kinematic and Dynamic Control

- **Collision Awareness**: The Differential IK solver does not natively account for self-collisions or obstacle avoidance. Future work involves adding Control Barrier Function (CBF) constraints to the QP formulation.
- **Full Dynamics Modeling**: The system is purely kinematic. Moving to a dynamic model (implementing gravity and inertia compensation) would significantly improve performance with heavy payloads.
- **Hybrid Singularities**: While SDLS handles mathematical singularities, the transition between discrete configuration branches (elbow flip) remains a focus for more advanced predictive "look-ahead" optimizations.
- **Jerk-Limited Motion**: The current controller provides smooth trajectories but is not inherently jerk-limited. This is because it solves a memoryless velocity-level QP. Achieving full jerk limiting would require either higher-order state constraints within the QP formulation or a dedicated time-domain filter applied as a post-processing stage.

4.4.3 Software and Integration

- **Automatic Hardware Detection**: The current automatic MCU detection system is known to fail intermittently. Manual port selection or retry logic is sometimes required to establish a stable communication link.
- **Control Loop Performance**: The high-level control loop, while stable, has several identified paths for optimization. Reducing computational overhead in the IK solver and improving thread synchronization would enable higher streaming frequencies and lower control latency.
- **Code-Base Modularity**: Continual refactoring to separate the core kinematic solvers from the GUI logic will enhance the portability of the controller to other robotic platforms.

Bibliography

[1] Buss, S. R., & Kim, J. S. (2005). Selectively Damped Least Squares for Inverse Kinematics. https://mathweb.ucsd.edu/~sbuss/ResearchWeb/ikmethods/SdlsPaper.pdf

[2] Buss, S. R., & Kim, J. S. (2005). A Selectively Damped Least Squares Method for Inverse Kinematics. *IEEE/ASME Transactions on Mechatronics*, 10(3), 312–320. https://ieeexplore.ieee.org/document/9757150

[3] Faverjon, B., & Tournassoud, P. (1987). A local based approach for path planning of manipulators with a high number of degrees of freedom.

[4] MathWorks. Manipulability index. Robotics System Toolbox. https://www.mathworks.com/help/robotics/ref/manipulabilityindex.html

[5] Stellato, B., Banjac, G., Goulart, P., Bemporad, A., & Boyd, S. (2020). OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4), 637–672.

[6] Ostermeier, D., Külz, J., & Althoff, M. (2025). Automatic Geometric Decomposition for Analytical Inverse Kinematics. *IEEE Robotics and Automation Letters*, 10(10), 9964–9971.

[7] BigTreeTech. TMC5160-V1.0 Stepper Motor Driver. Product Page. https://biqu.equipment/products/bigtreetech-tmc5160-v1-0-driver-spi-mode-silent-high-precision-stepstick-stepper-m

Appendix A

Source Code Repositories

The source code for the various components of this project is hosted on GitHub.

- **Control Board Firmware**: `[GitHubLinkPlaceholder]`
- **Robot Controller Software**: `[GitHubLinkPlaceholder]`
- **Visualization UI**: `[GitHubLinkPlaceholder]`